

SpaceY Launch Cost Prediction Analysis

Predicting Falcon 9 First Stage Landing Success

IBM Data Science Professional Certificate — Capstone Project

Ryan Ordonez

GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

Executive Summary

Methods Used:

- Data collected via SpaceX REST API (v4) and Wikipedia web scraping using BeautifulSoup.
- Data wrangled: missing value imputation, target variable (Class) engineering.
- EDA performed with Matplotlib, Seaborn visualizations and 10 SQL queries in SQLite.
- Interactive analytics built with Folium maps and a Plotly Dash dashboard.
- Four classification models trained with GridSearchCV (cv=10): Logistic Regression, SVM, Decision Tree, KNN.

Key Results:

- All four models achieved 83.33% test accuracy on 18 test samples.
- Decision Tree achieved the highest cross-validation score: 88.75%.
- Landing success rate improved from 0% (2010) to over 80% (2020).
- KSC LC-39A is the most successful launch site; ES-L1, GEO, HEO, SSO orbits show 100% success.
- Successful landing prediction enables competitive bidding at ~\$62M vs ~\$165M per launch.

Table of Contents

1. Executive Summary
2. Introduction
3. Data Collection — SpaceX API (Methodology)
4. Data Collection — Web Scraping (Methodology)
5. Data Wrangling (Methodology)
6. EDA with Data Visualization (Methodology)
7. EDA with SQL (Methodology)
8. Interactive Visual Analytics (Methodology)
9. Predictive Analysis (Methodology)
10. EDA Visualization Results
11. EDA with SQL Results
12. Interactive Maps — Folium Results
13. Interactive Dashboard — Plotly Dash Results
14. Predictive Analysis Results
15. Conclusion & Innovative Insights
16. Appendix

Introduction

Project Background

SpaceX has disrupted the rocket launch industry by successfully landing and reusing the Falcon 9 first stage, reducing launch cost from ~\$165M to ~\$62M.

Problem Statement

SpaceY, a fictional competitor, wants to predict whether the Falcon 9 first stage will land successfully. This prediction directly determines launch cost:

- **Successful landing** → ~\$62M (reusable)
- **Failed landing** → ~\$165M (expendable)

Methodology Overview

1. Data Collection (SpaceX API + Web Scraping)
2. Data Wrangling (cleaning, Class variable)
3. EDA with Visualization (Matplotlib, Seaborn)
4. EDA with SQL (SQLite, 10 queries)
5. Interactive Analytics (Folium maps, Plotly Dash)
6. Predictive Modeling (LR, SVM, DT, KNN)

Dataset: 90 Falcon 9 launches (2010–2020)

Target: Class (1 = landed, 0 = failed/no attempt)

Features: 83 columns after one-hot encoding

Data Collection Methodology — SpaceX REST API

API Endpoint: api.spacexdata.com/v4

Data Collection Process (Flowchart):

```
| GET /v4/launches/past |  
| (Retrieve all past launches) |
```



```
| Filter: rocket == Falcon 9 ID |  
| (5e9d0d95eda69973a809d1ec) |
```



```
| For each launch, resolve IDs: |  
| • GET /v4/rockets/{id} |  
| • GET /v4/launchpads/{id} |  
| • GET /v4/payloads/{id} |  
| • GET /v4/cores/{id} |
```



GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

```
| Build flat DataFrame (90 × 17) |
```

Key Columns Extracted:

FlightNumber, Date, BoosterVersion
PayloadMass, Orbit, LaunchSite
Outcome (landing success + type)
Flights, GridFins, Reused, Legs
LandingPad, Block, ReusedCount, Serial
Longitude, Latitude

Outcome Column Format:

"True ASDS" → successful drone ship landing
"True RTLS" → successful ground pad landing
"False Ocean" → failed ocean landing
"None None" → no landing attempt

Tools: Python requests library

Output: data/dataset_part_1.csv (90 rows × 17 columns)

[Notebook: notebooks/01_spacex_api_collection.ipynb](#)

Data Collection Methodology — Web Scraping

Source: Wikipedia — List of Falcon 9 Launches

(Static HTML snapshot, June 9 2021)

Web Scraping Process (Flowchart):

```
| requests.get(wikipedia_url) |  
| (Fetch static HTML page)   |
```



```
| BeautifulSoup(html, html.parser) |  
| Parse HTML structure              |
```



```
| Find all <table> elements         |  
| Extract <th> headers → columns   |
```



```
| Iterate <tr> rows, extract cells |  
| Helper funcs: date_time,        |  
| booster_version, landing_status, |
```

GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

Columns Scraped:

Flight No., Date, Time (UTC)
Version Booster, Launch site
Payload, Payload mass, Orbit
Customer, Launch outcome
Booster landing

Helper Functions:

- `date_time()` — parse launch date/time
- `booster_version()` — extract booster info
- `landing_status()` — parse landing outcome
- `get_mass()` — extract numeric payload mass
- `extract_column_from_header()` — get headers

Tools: requests, BeautifulSoup4

Output: data/spacex_web_scraped.csv

Notebook: [notebooks/02_web_scraping.ipynb](#)

Data Wrangling Methodology

Data Cleaning & Processing Steps:

Step 1: Load dataset_part_1.csv (90 rows × 17 cols)

Step 2: Identify missing values

- LandingPad: 28.9% missing (26 nulls)
- PayloadMass: minor nulls → imputed with mean

Step 3: Analyze launch frequency

- Count launches per site
- Count launches per orbit type

Step 4: Analyze landing outcome categories

- True ASDS, True RTLS → Success
- False Ocean, None None → Failure

Step 5: Engineer target variable — Class

Target Variable: Class

Class = 1 (Successful Landing)

- Outcome starts with 'True'
- True ASDS, True RTLS, True Ocean
- Count: 60 launches (66.7%)

Class = 0 (Failed / No Attempt)

- Outcome starts with 'False' or 'None'
- False ASDS, False Ocean, None None
- Count: 30 launches (33.3%)

Output: data/dataset_part_2.csv (90 rows × 18 cols)

Notebook: [notebooks/03_data_wrangling.ipynb](#)

EDA with Data Visualization Methodology

Charts Created and Their Purpose:

1. Flight Number vs Launch Site (scatter)
→ See which sites used over time
2. Flight Number vs Payload Mass (scatter)
→ Relationship between payload and success
3. Payload Mass vs Launch Site (scatter)
→ Payload capacity per site
4. Success Rate by Orbit Type (bar chart)
→ Which orbits have highest success
5. Flight Number vs Orbit Type (scatter)
→ Orbit selection evolution over time
6. Payload Mass vs Orbit Type (scatter)
→ Payload requirements per orbit
7. Success Rate by Launch Site (bar chart)
→ Compare site reliability
8. Yearly Success Trend (line chart)
→ Track improvement over time

Feature Engineering:

- One-hot encoding of categorical columns:
Orbit, LaunchSite, LandingPad, Serial
- Cast all features to float64
- Result: 90 rows × 83 feature columns

Tools: Matplotlib, Seaborn

Output:

- 8 PNG charts saved to images/
- data/dataset_part_3.csv (engineered features)

Notebook: [notebooks/04_eda.ipynb](https://github.com/RyanOrdonez/spacey-capstone/blob/master/notebooks/04_eda.ipynb)

EDA with SQL Methodology

SQL Queries Performed (10 queries in SQLite):

Q1: SELECT DISTINCT Launch_Site — unique launch sites

Q2: SELECT * WHERE Launch_Site LIKE 'CCA%' LIMIT 5 — filter by site prefix

Q3: SELECT SUM(PAYLOAD_MASS_KG_) WHERE Customer = 'NASA (CRS)' — total payload for customer

Q4: SELECT AVG(PAYLOAD_MASS_KG_) WHERE Booster_Version LIKE 'F9 v1.1%' — avg payload for booster

Q5: SELECT MIN(Date) WHERE Landing_Outcome = 'Success (ground pad)' — first ground pad landing

Q6: SELECT Booster_Version WHERE Landing_Outcome = 'Success (drone ship)' AND PAYLOAD 4000-6000

Q7: SELECT Mission_Outcome, COUNT(*) GROUP BY Mission_Outcome — mission outcome counts

Q8: SELECT Booster_Version WHERE PAYLOAD_MASS = MAX — max payload boosters

Q9: SELECT * WHERE Landing_Outcome = 'Failure (drone ship)' AND Date LIKE '2015%' — 2015 failures

Q10: SELECT Landing_Outcome, COUNT(*) WHERE Date BETWEEN 2010-2017 ORDER BY COUNT — rankings

Database: SQLite | Table: SPACEXTABLE | Loaded from dataset_part_2.csv

SQL script: sql/spacex_queries.sql

Notebook: [notebooks/05_sql_analysis.ipynb](#)

Interactive Visual Analytics Methodology

Folium Interactive Maps:

Map 1: Launch Site Map

- Mark all 4 launch sites with circles
- Color-code markers: green=success, red=fail
- MarkerCluster for interactive exploration
- Popup labels with site name + class

Map 2: KSC LC-39A Proximity Analysis

- Calculate distance to coastline (~1.2 km)
- Calculate distance to highway (~7.1 km)
- Calculate distance to railway (~5.7 km)
- Calculate distance to city (~52 km)
- Draw lines and markers on map

Output: output/launch_site_map.html

Output: output/ksc_proximity_map.html

Notebook: [notebooks/06_interactive_visuals.ipynb](#)

Plotly Dash Dashboard:

Dashboard Components:

1. Dropdown: Select launch site (or All Sites)
2. Pie Chart: Success vs Failure for site
3. Range Slider: Filter by payload mass (0-10,000 kg)
4. Scatter Plot: Payload vs Outcome by Booster

Implementation:

- Framework: Plotly Dash
- Two reactive `@app.callback` decorators
- Input: site dropdown + payload slider
- Output: pie chart + scatter chart
- Data: `spacex_launch_dash.csv` (IBM Cloud)

Run: `python dash/spacex_dash_app.py`

Open: `http://localhost:8050`

Script: `dash/spacex_dash_app.py`

Predictive Analysis Methodology

Data Preparation:

- Target Y from dataset_part_2.csv (Class column)
- Features X from dataset_part_3.csv (83 cols)
- StandardScaler normalization
- Train/Test split: 80/20 (random_state=2)
- Train: 72 samples | Test: 18 samples

Models Trained:

1. Logistic Regression

C: [0.01, 0.1, 1], penalty: l2

2. Support Vector Machine (SVM)

kernel: [linear, rbf, poly, sigmoid]

C: logspace(-3,3,5), gamma: logspace(-3,3,5)

3. Decision Tree Classifier

criterion: [gini, entropy], max_depth: [2-18]

4. K-Nearest Neighbors (KNN)

n_neighbors: [1-10], p: [1,2]

Hyperparameter Tuning:

- GridSearchCV with cv=10
- Exhaustive search over parameter grids
- Best params selected by CV accuracy

Evaluation Metrics:

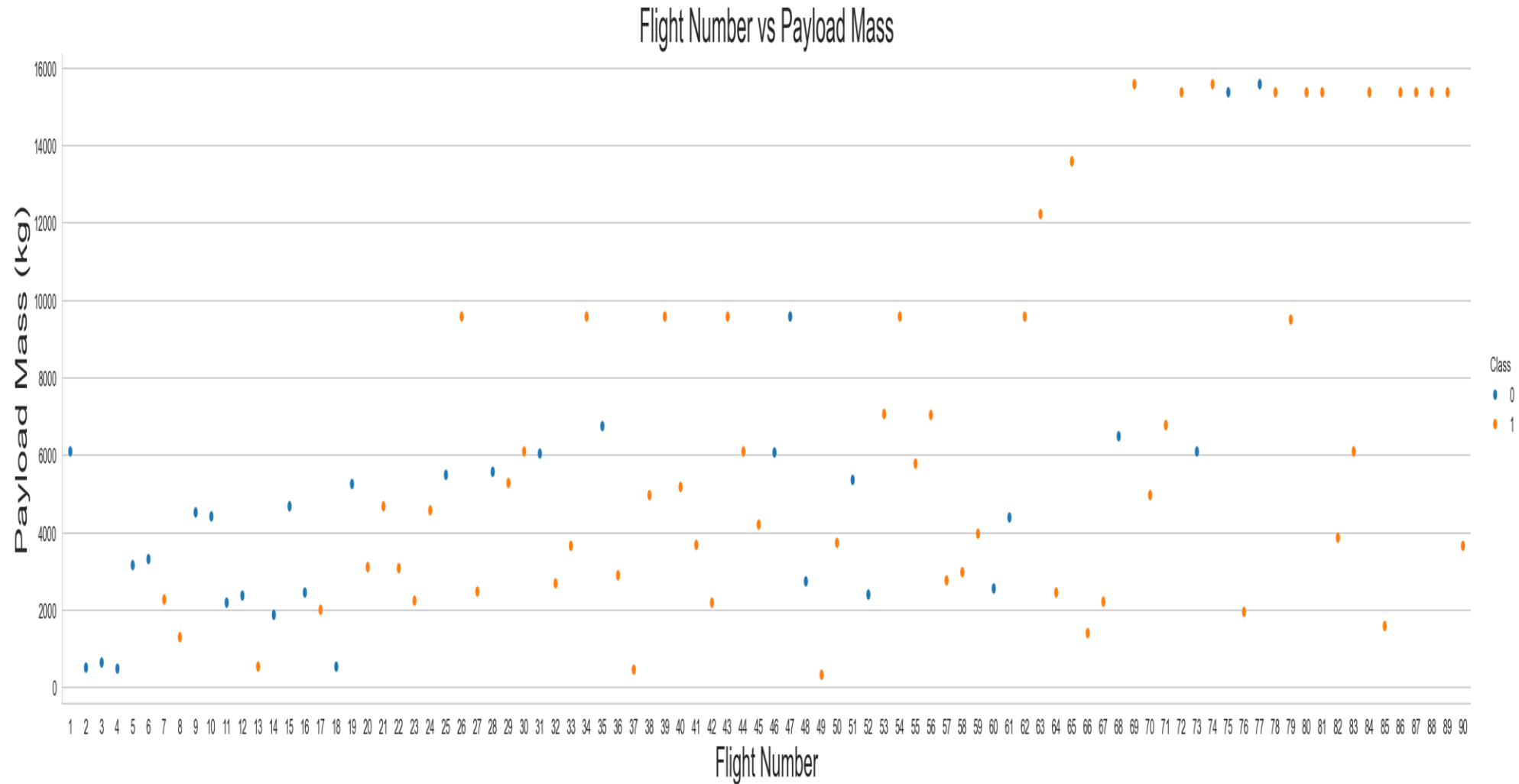
- Cross-validation accuracy (10-fold)
- Test set accuracy
- Confusion matrix (True/False Pos/Neg)
- Side-by-side model comparison bar chart

Tools: scikit-learn

GridSearchCV, StandardScaler,
confusion_matrix, accuracy_score

Notebook: [notebooks/07_modeling.ipynb](#)

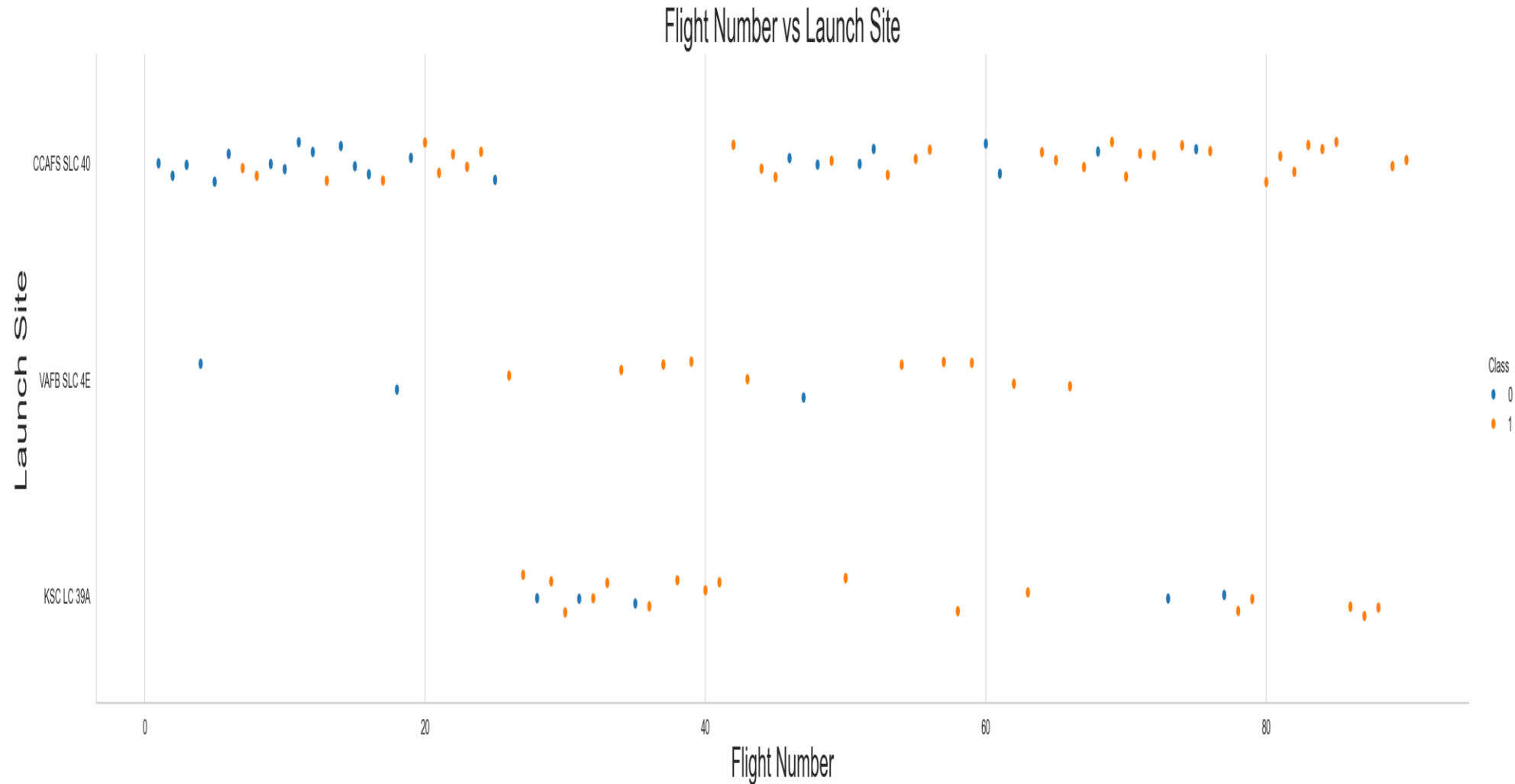
EDA Results — Flight Number vs Payload Mass (Scatter Plot)



Higher flight numbers correlate with more successful landings. Payload mass alone does not determine landing outcome.

GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

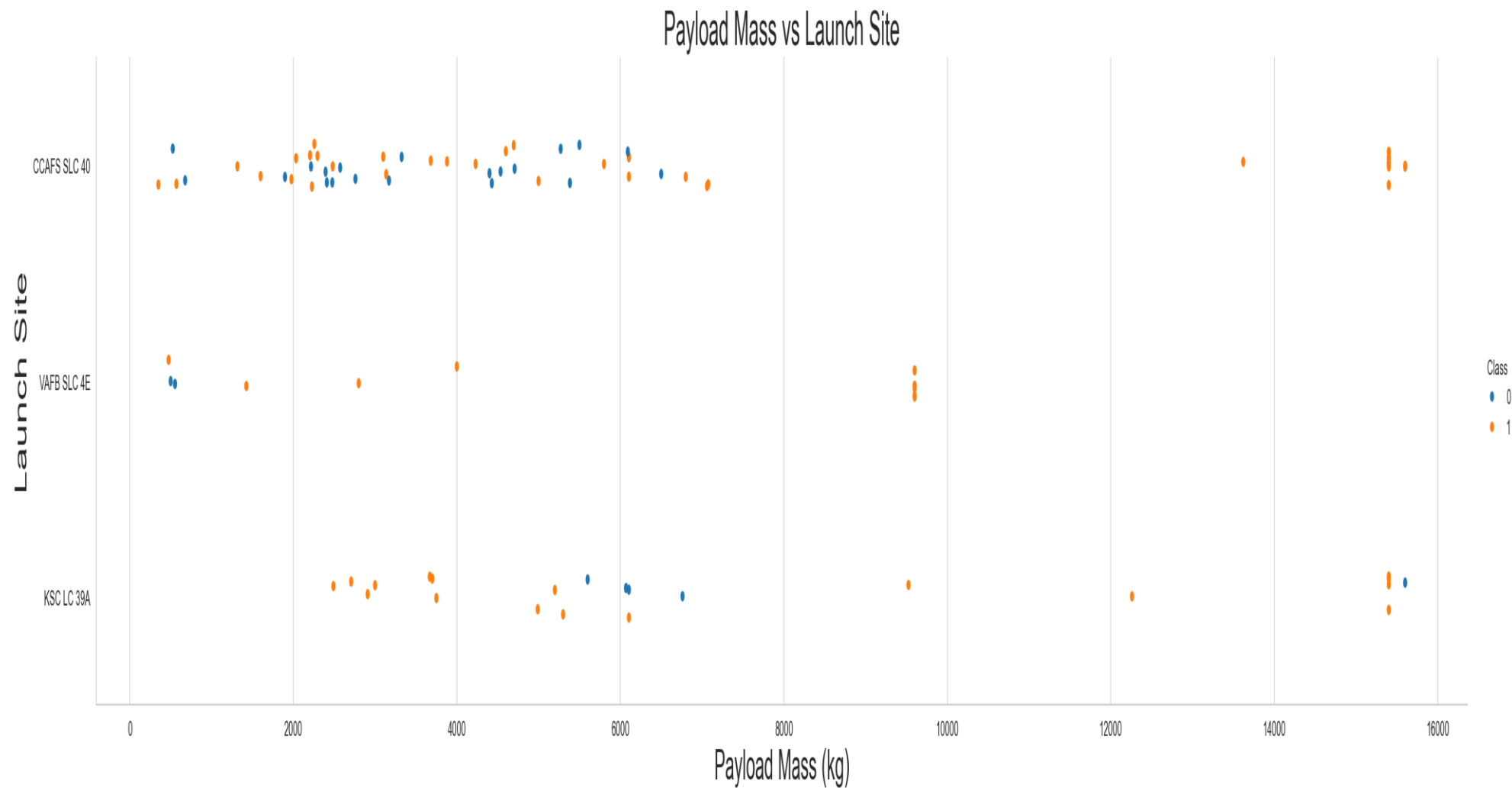
EDA Results — Flight Number vs Launch Site (Scatter Plot)



KSC LC-39A has the most launches and highest success concentration at higher flight numbers. VAFB SLC-4E used mainly for polar orbits.

GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

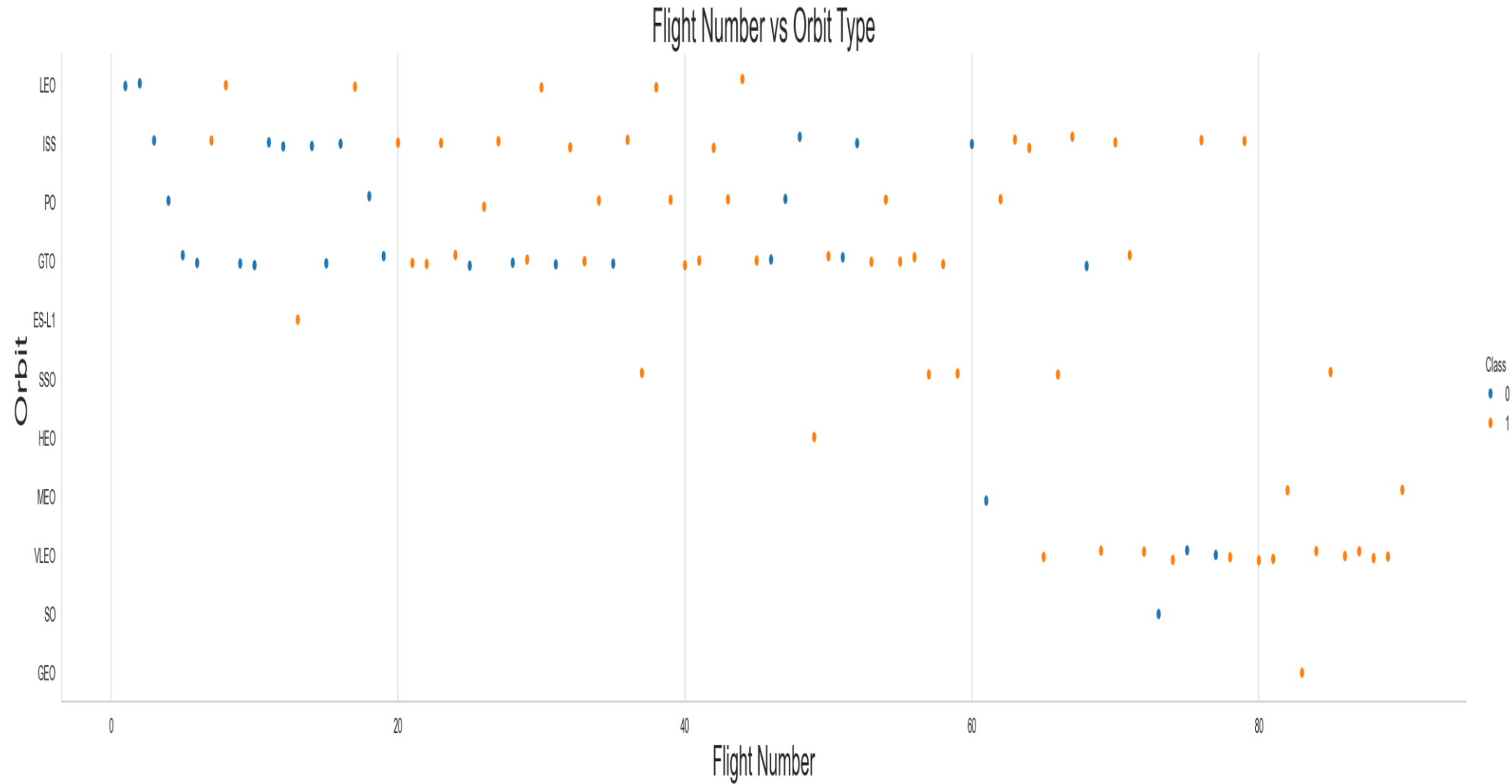
EDA Results — Payload Mass vs Launch Site (Scatter Plot)



KSC LC-39A handles the heaviest payloads (up to 15,600 kg). All sites show successful launches across payload ranges.

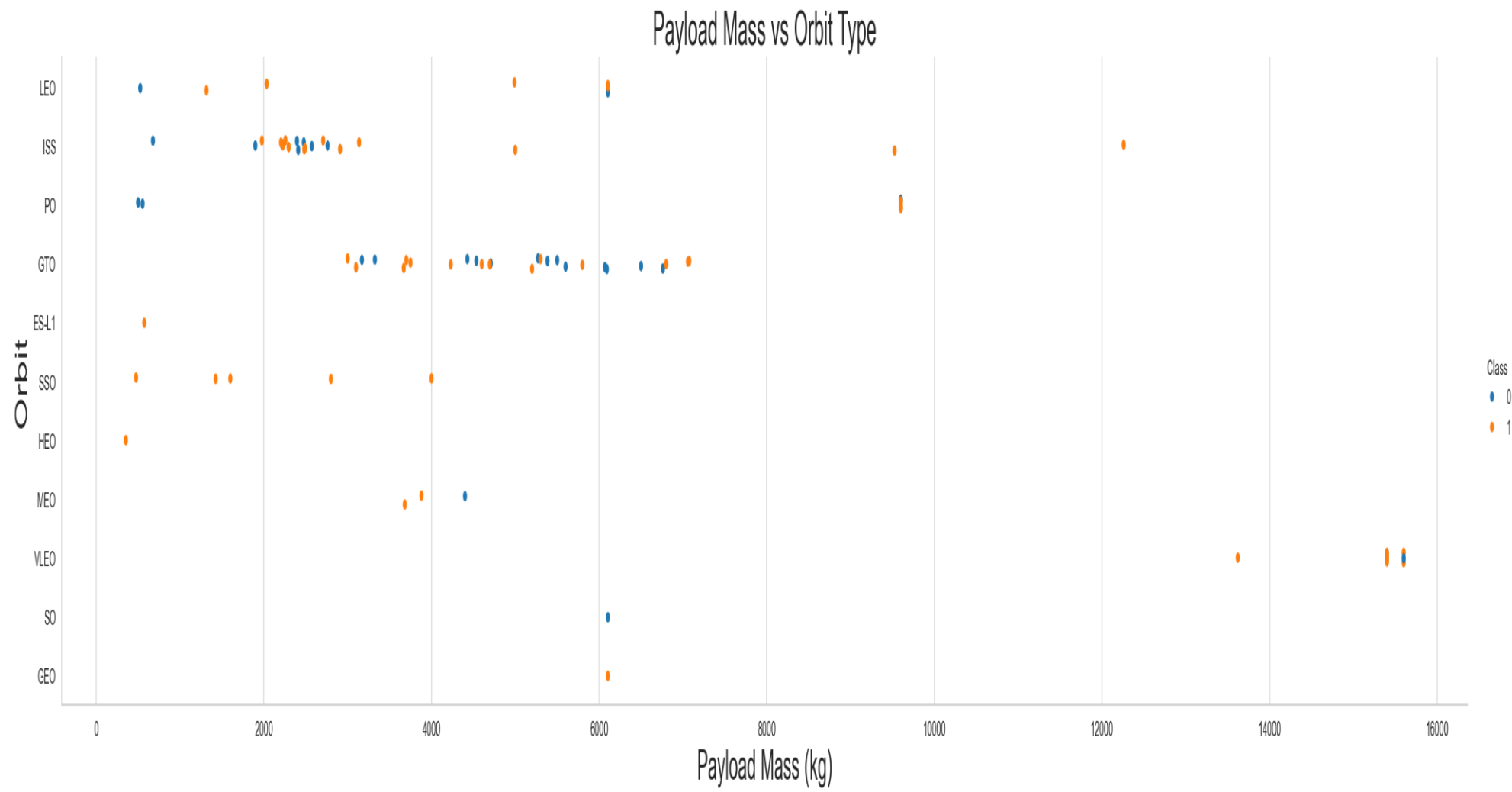
GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

EDA Results — Flight Number vs Orbit Type (Scatter Plot)



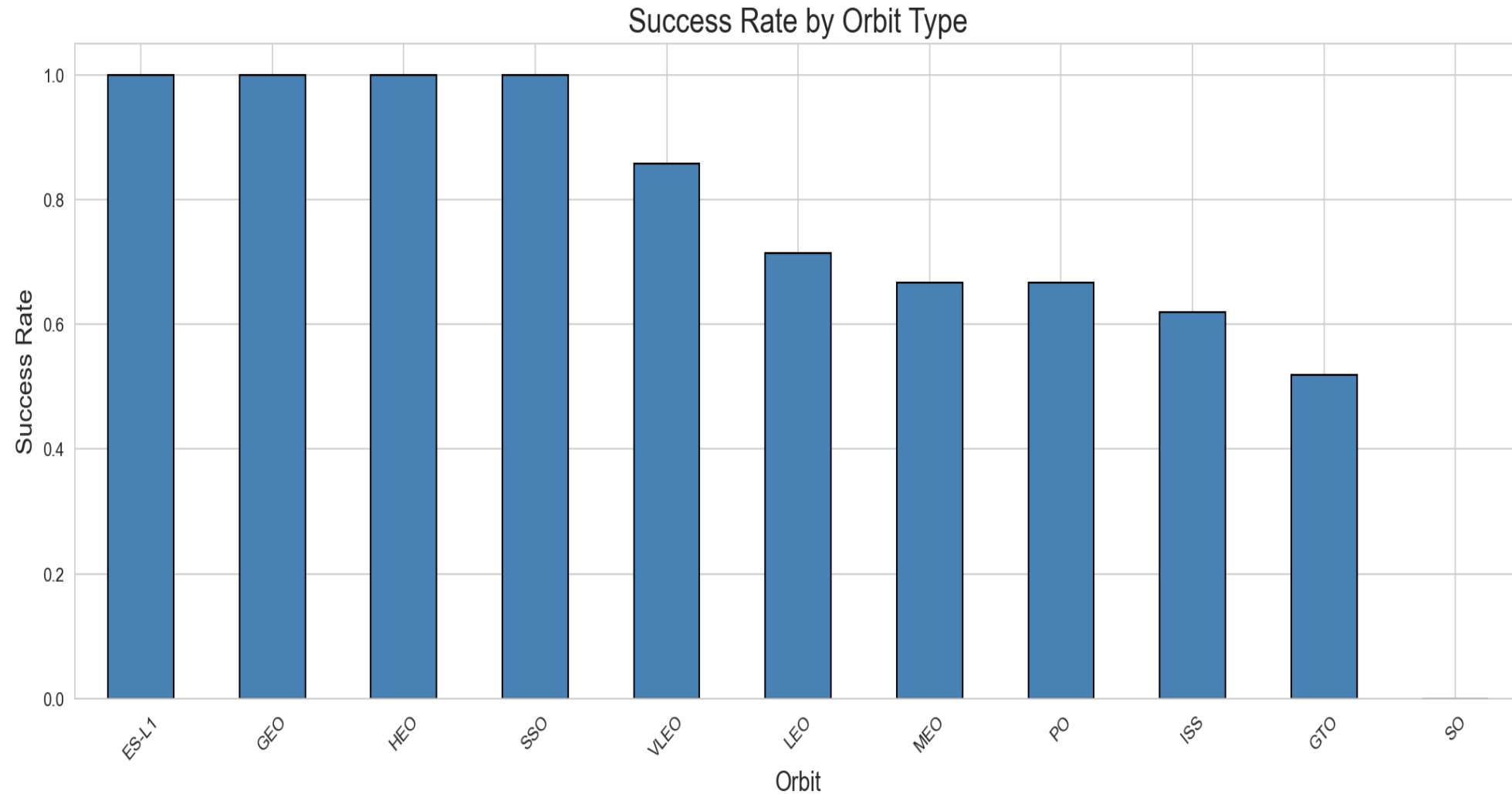
LEO and ISS are the most common orbits in early flights. GTO missions appear throughout the timeline.
GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

EDA Results — Payload Mass vs Orbit Type (Scatter Plot)



GTO and GEO orbits typically require higher payload masses. LEO/ISS orbits show wider payload mass ranges.
GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

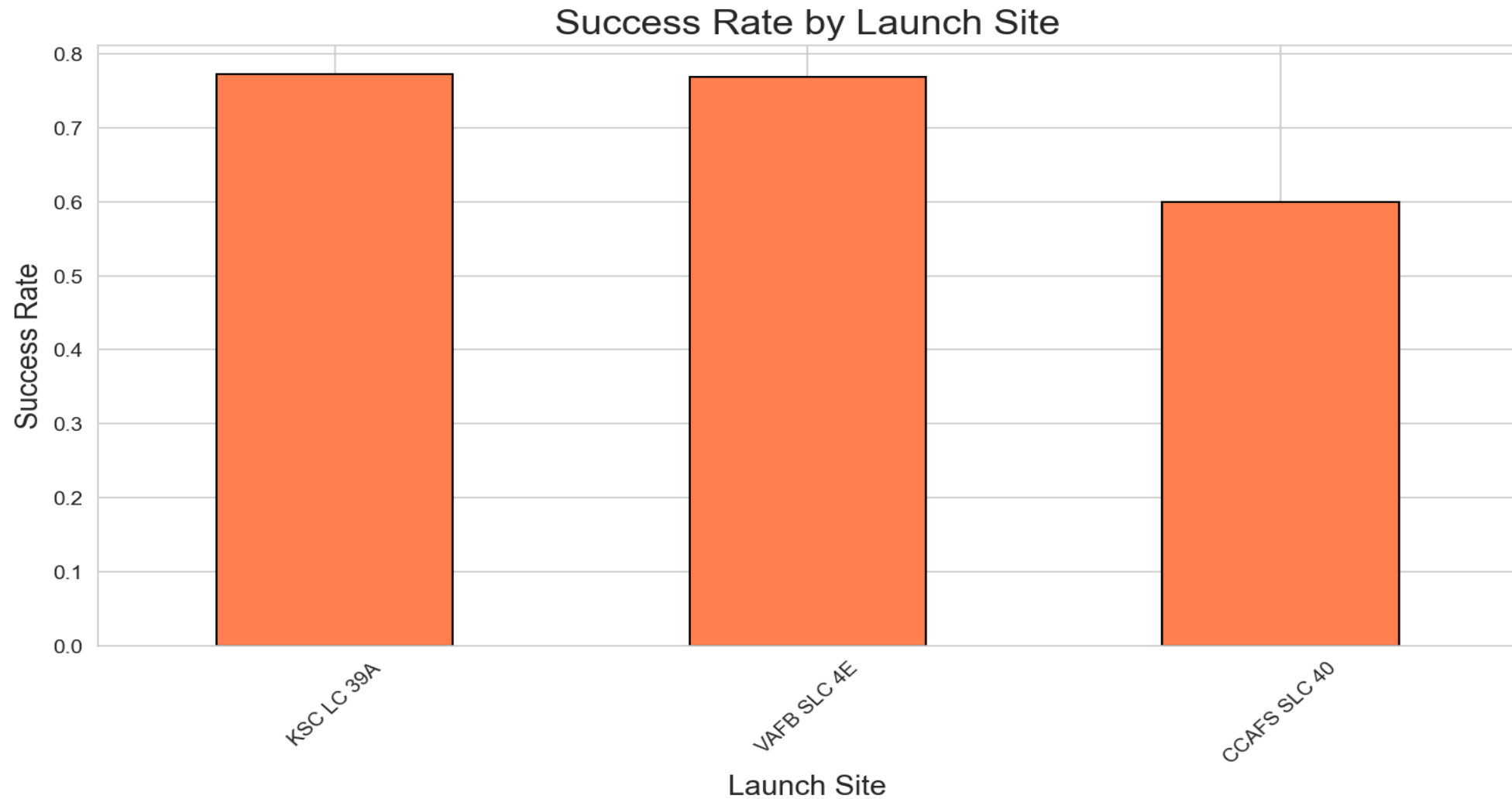
EDA Results — Success Rate by Orbit Type (Bar Chart)



ES-L1, GEO, HEO, and SSO orbits show 100% landing success. GTO has mixed results due to higher energy requirements.

GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

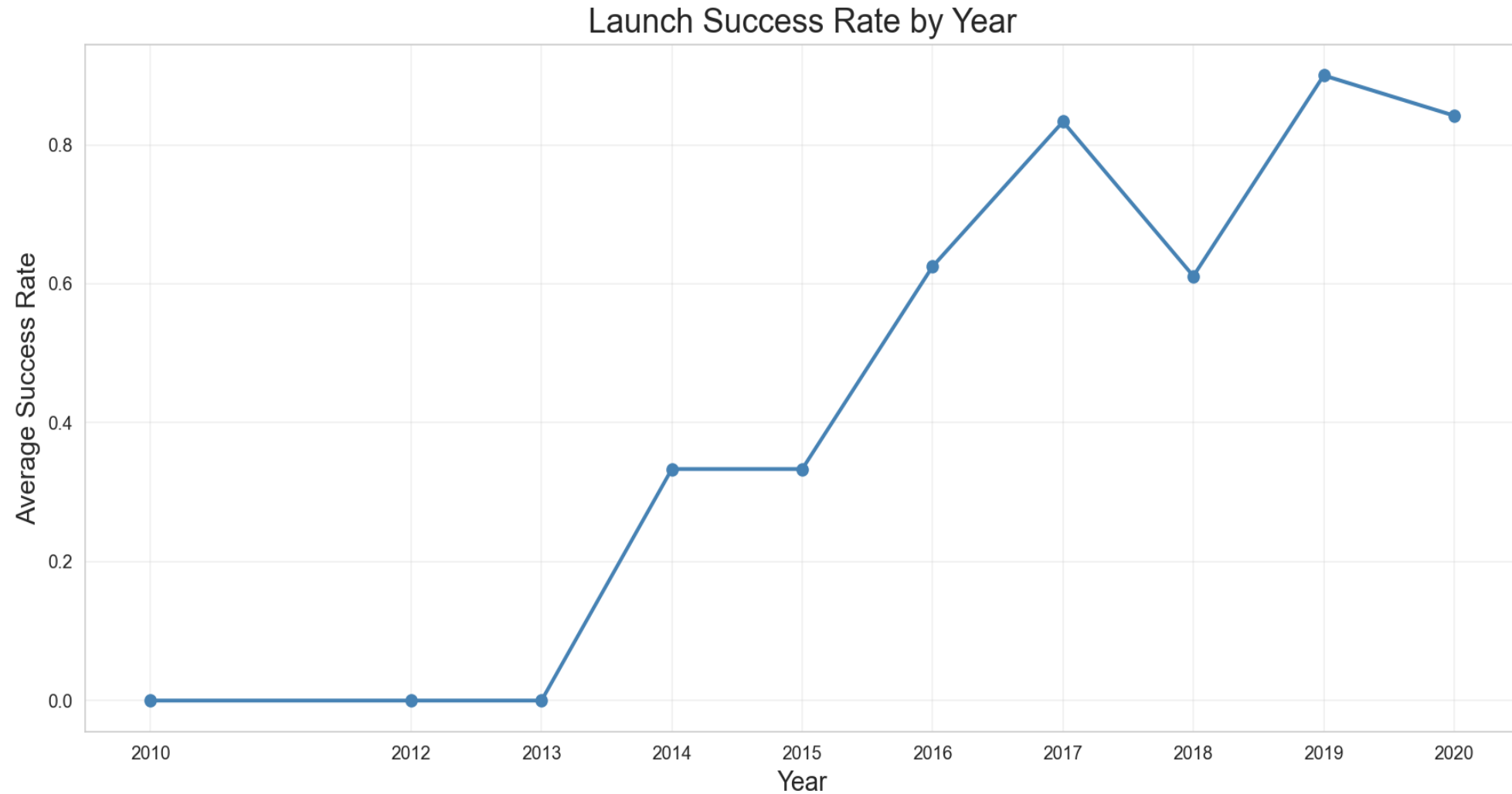
EDA Results — Success Rate by Launch Site (Bar Chart)



KSC LC-39A leads with the highest success rate. All sites show improvement over time.

GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

EDA Results — Yearly Success Trend (Line Chart)



Success rate improved from 0% in 2010 to over 80% by 2019–2020, demonstrating SpaceX's engineering learning curve.

GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

EDA with SQL Results — Launch Sites, Payloads, Success Rates

Q1: Unique Launch Sites

CCAFS LC-40, CCAFS SLC-40, KSC LC-39A, VAFB SLC-4E

Q2: Records with Launch Site LIKE 'CCA%'

Cape Canaveral sites returned (5 sample rows)

Q3: NASA (CRS) Total Payload

Total: 45,596 kg across all CRS missions

Q4: F9 v1.1 Average Payload Mass

Average: 2,928.4 kg

Q5: First Successful Ground Pad Landing

Date: 2015-12-22 (Orbcomm-OG2 mission)

Q6: Drone Ship Success (4000-6000 kg)

F9 FT B1021.2, F9 FT B1031.2

Q7: Mission Outcome Counts

Success: 98 | Failure: 1 | Partial: 1

Q8: Max Payload Boosters

F9 B5 series (Block 5 — latest generation)

Q9: 2015 Drone Ship Failures

Jan 2015 (CRS-5) and Apr 2015 (CRS-6)

Q10: Landing Outcome Rankings (2010–2017)

No Attempt: 21 | Failure (drone ship): 9

Success (drone ship): 5 | Failure (ASDS): 2

Success (ground pad): 3 | Controlled (ocean): 5

EDA with SQL Results — Time Analysis & Rankings

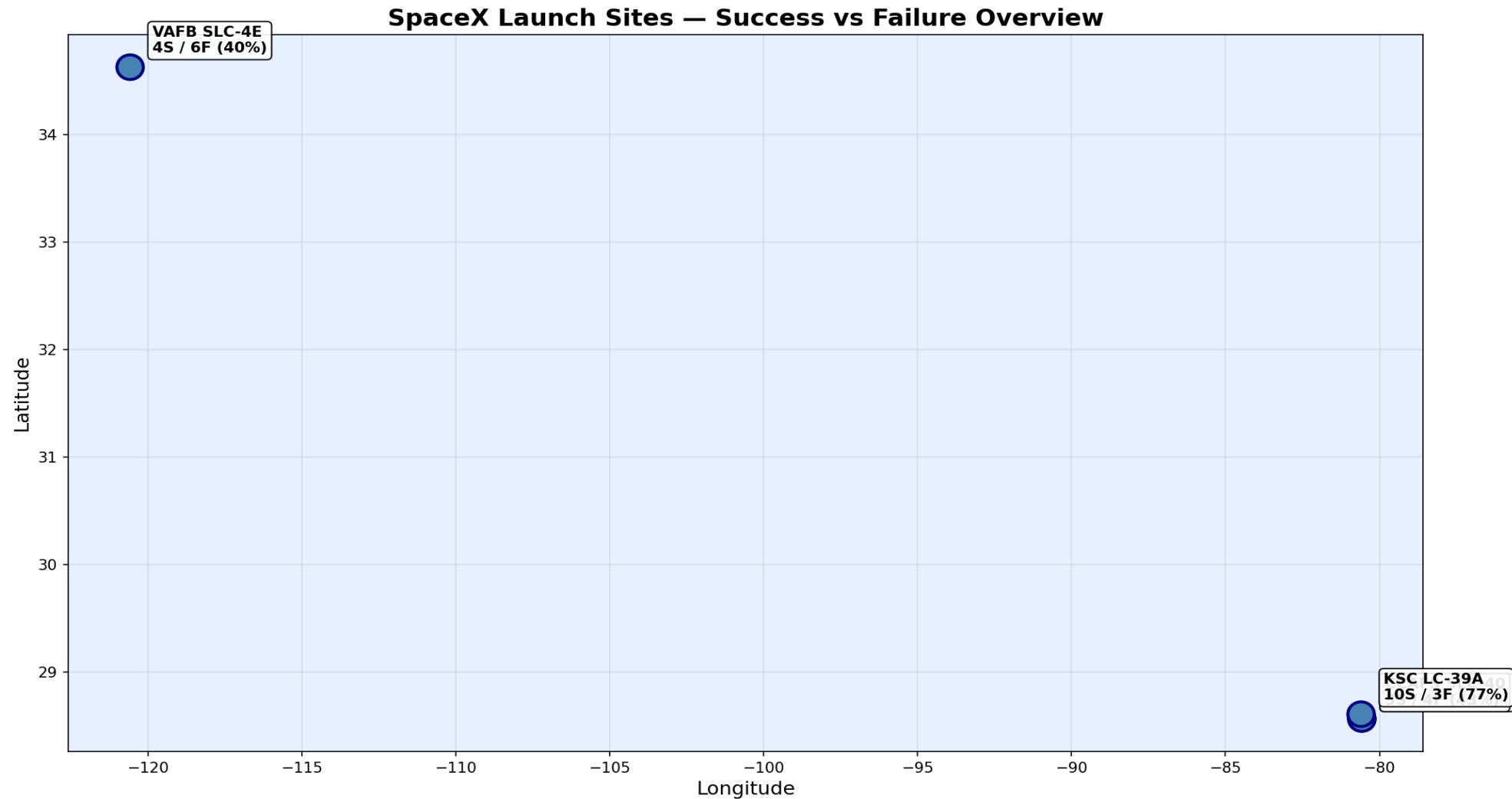
Key Time-Based Insights from SQL Analysis:

- First successful ground pad landing: December 22, 2015
This was the Orbcomm-OG2 mission from CCAFS LC-40 — a milestone in reusability.
- First successful drone ship landing: April 8, 2016
CRS-8 mission — opened up recovery for high-energy GTO launches.
- 2015 was the turning point: First landing successes after years of failed attempts.
- By 2017: Success rate exceeded 50% for the first time.

Landing outcome distribution (2010–2017):

Landing Outcome	Count
No Attempt	21
Failure (drone ship)	9
Controlled (ocean)	5
Success (drone ship)	5
Success (ground pad)	3
Failure (ADS)	2
Uncontrolled (ocean)	2

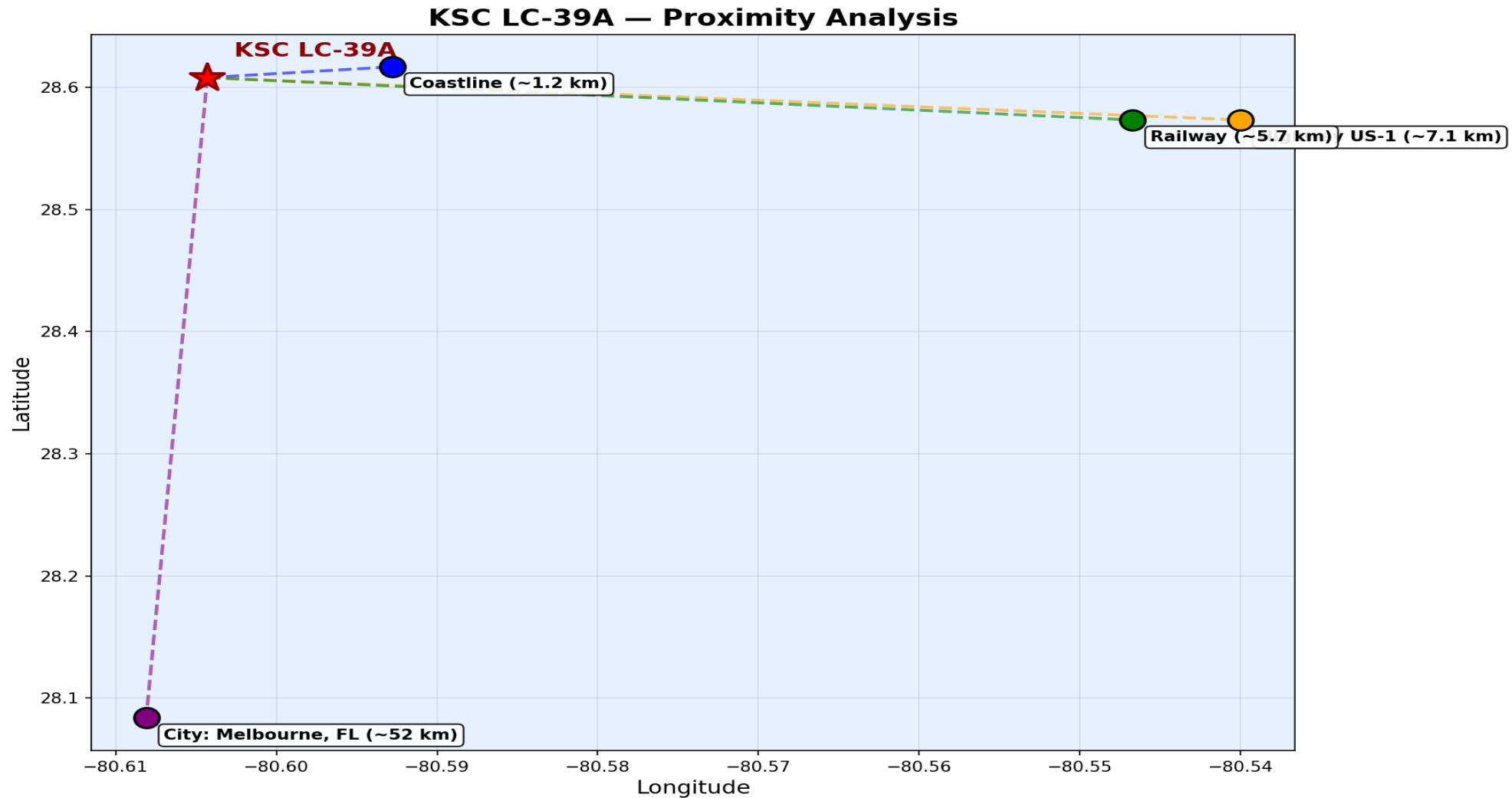
Folium Map Results — Launch Site Markers & Launch Records



All 4 SpaceX launch sites mapped with success/failure counts. Interactive version: [output/launch_site_map.html](https://github.com/RyanOrdonez/spacey-capstone)

GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

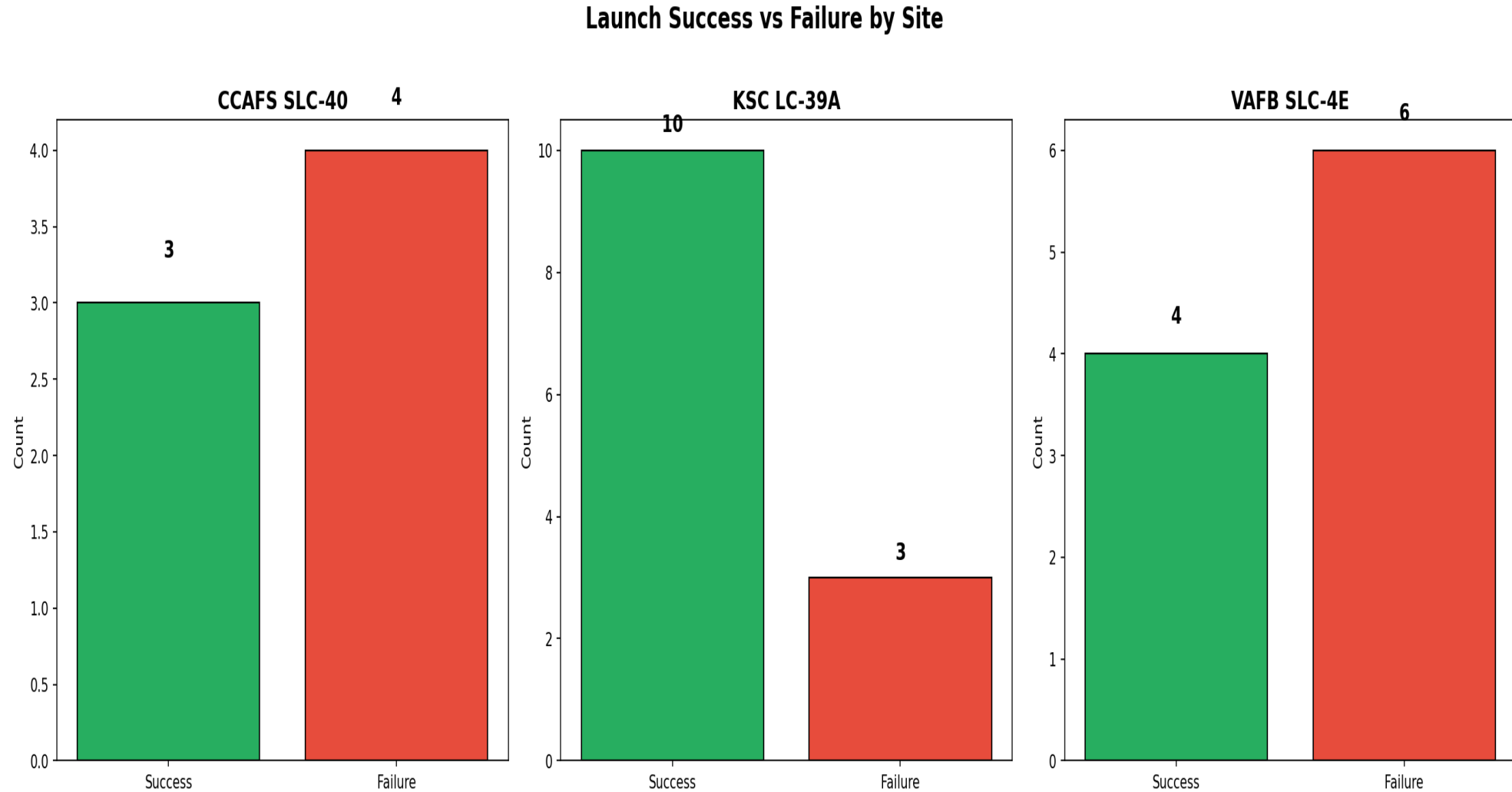
Folium Map Results — KSC LC-39A Proximity Analysis



KSC LC-39A is ~1.2 km from coast, ~7.1 km from highway, ~5.7 km from railway, ~52 km from nearest city. Interactive: output/ksc_proximity_map.html

GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

Folium Map Results — Success vs Failure by Launch Site

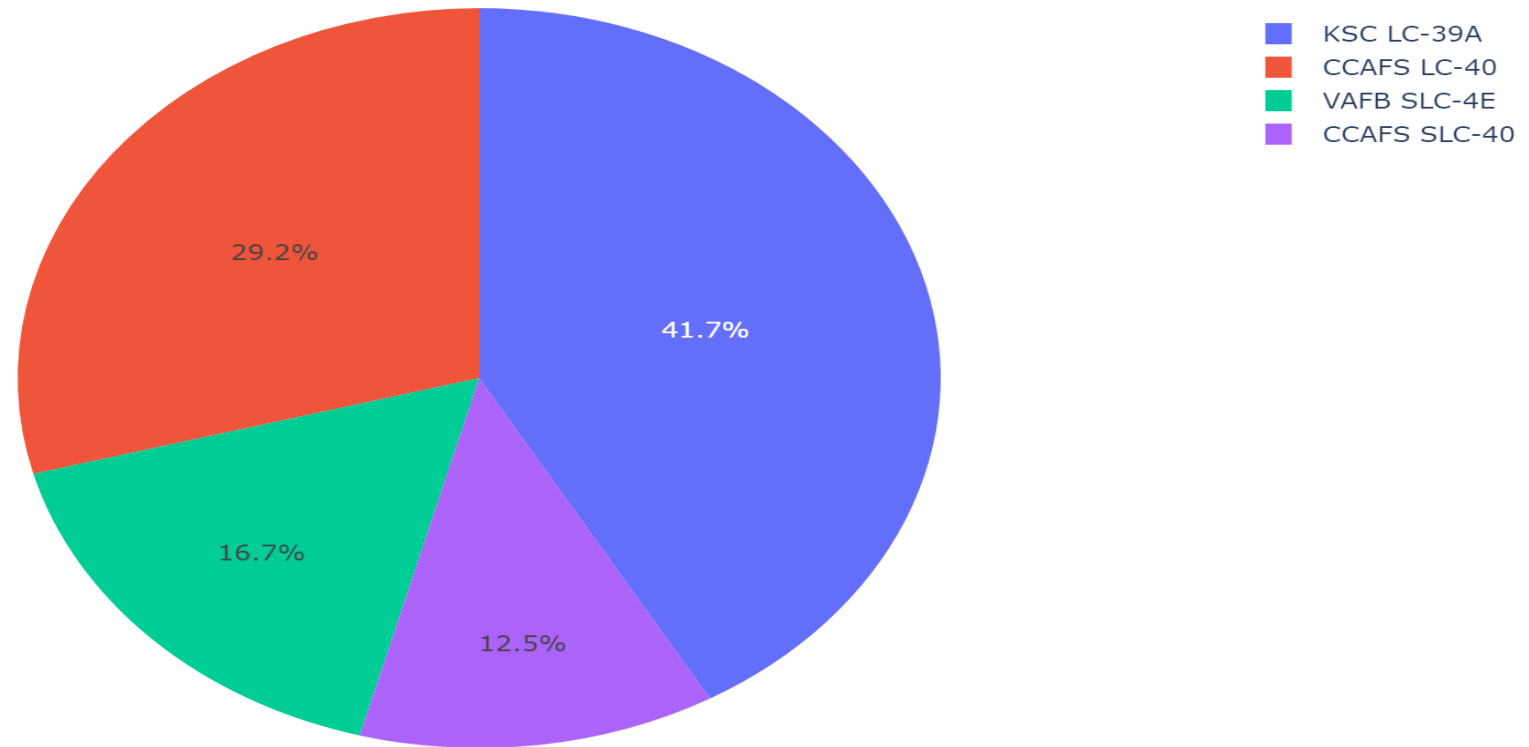


Success/failure breakdown per site. All sites are coastal for safety. KSC LC-39A has the most launches and highest success count.

GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

Plotly Dash Results — Success Pie Chart (All Sites)

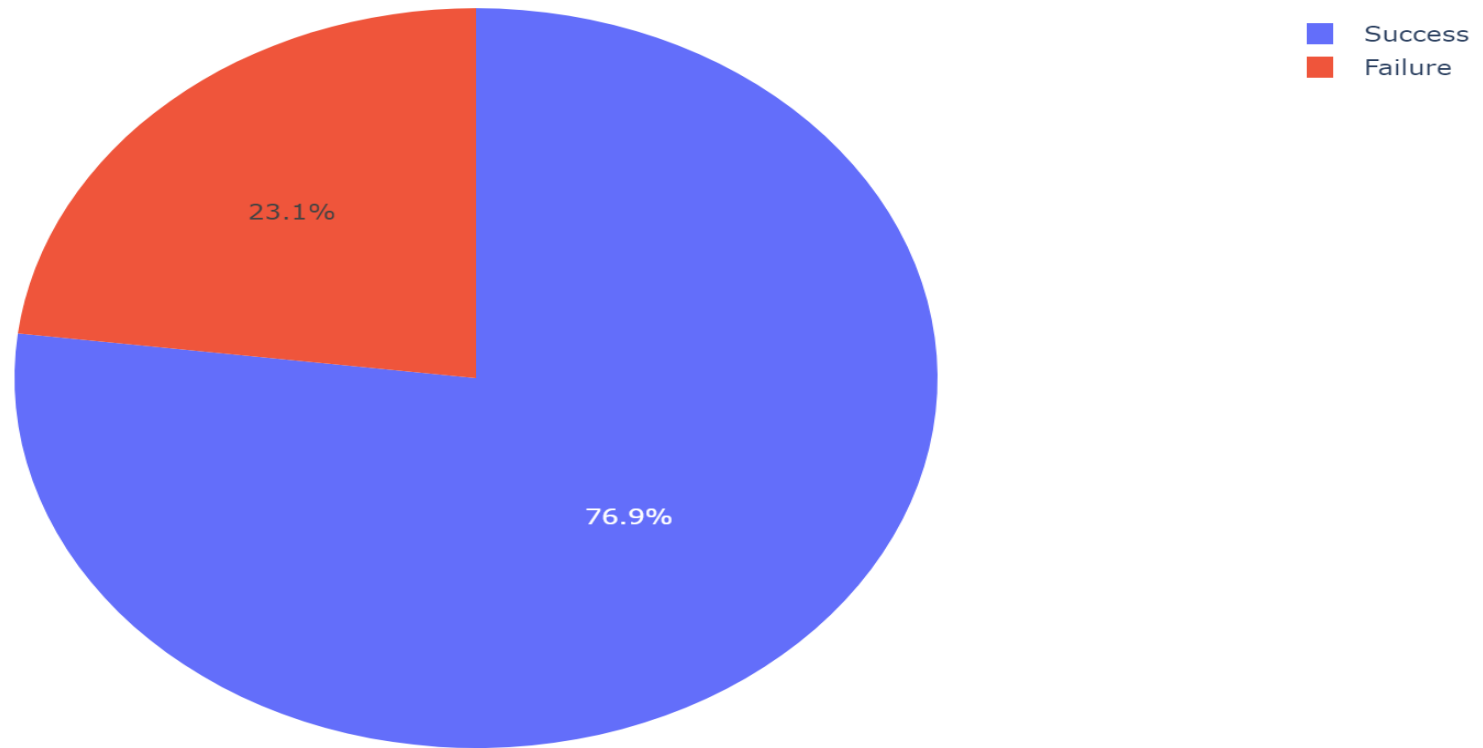
Total Successful Launches by Site



Pie chart showing total successful launches distributed across all sites. KSC LC-39A contributes the most successes.
GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

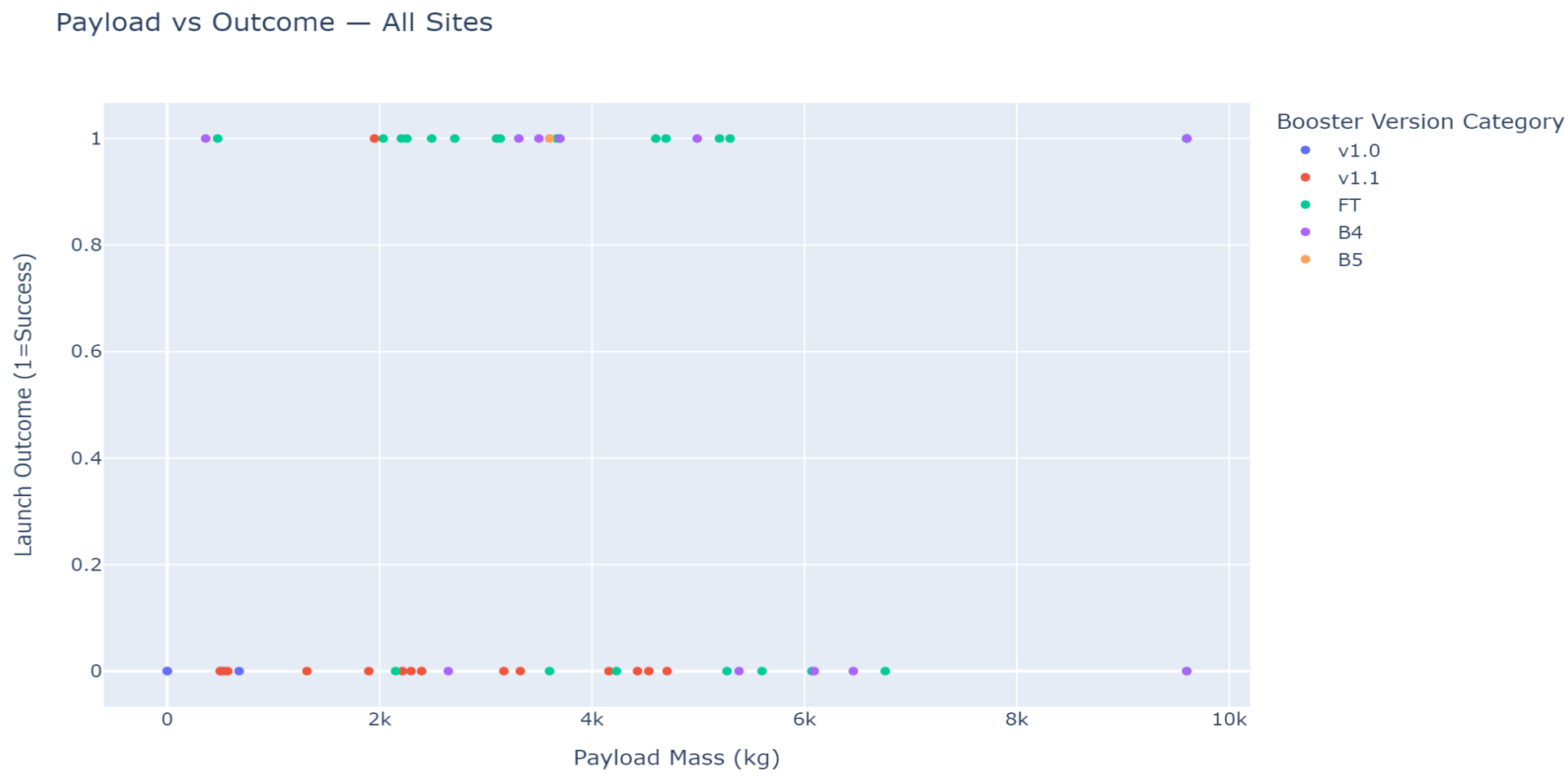
Plotly Dash Results — Success vs Failure Pie Chart (KSC LC-39A)

Success vs Failure — KSC LC-39A



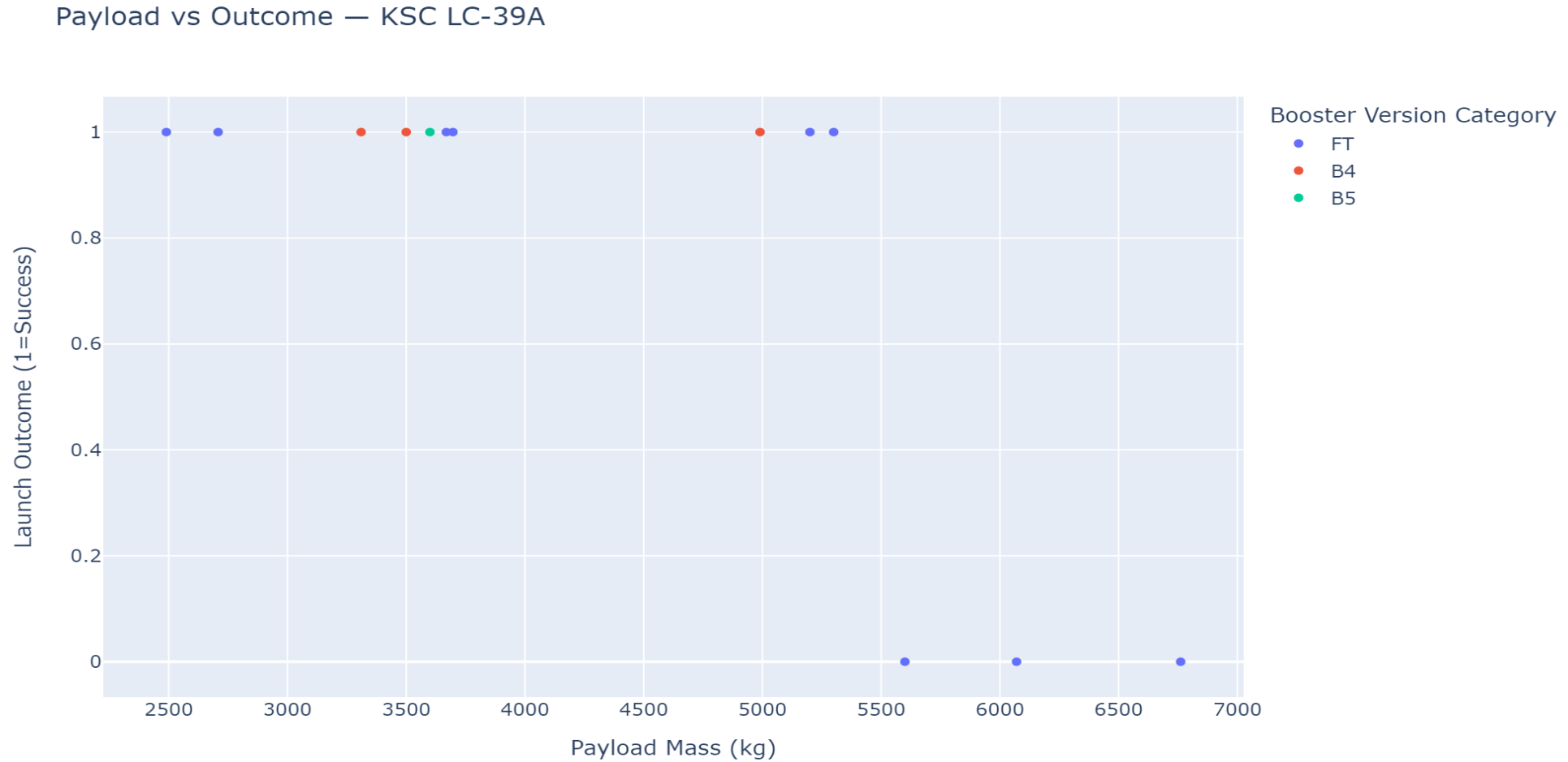
KSC LC-39A: 77% success rate — the most reliable launch site in the dataset.
GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

Plotly Dash Results — Payload vs Outcome Scatter (All Sites)



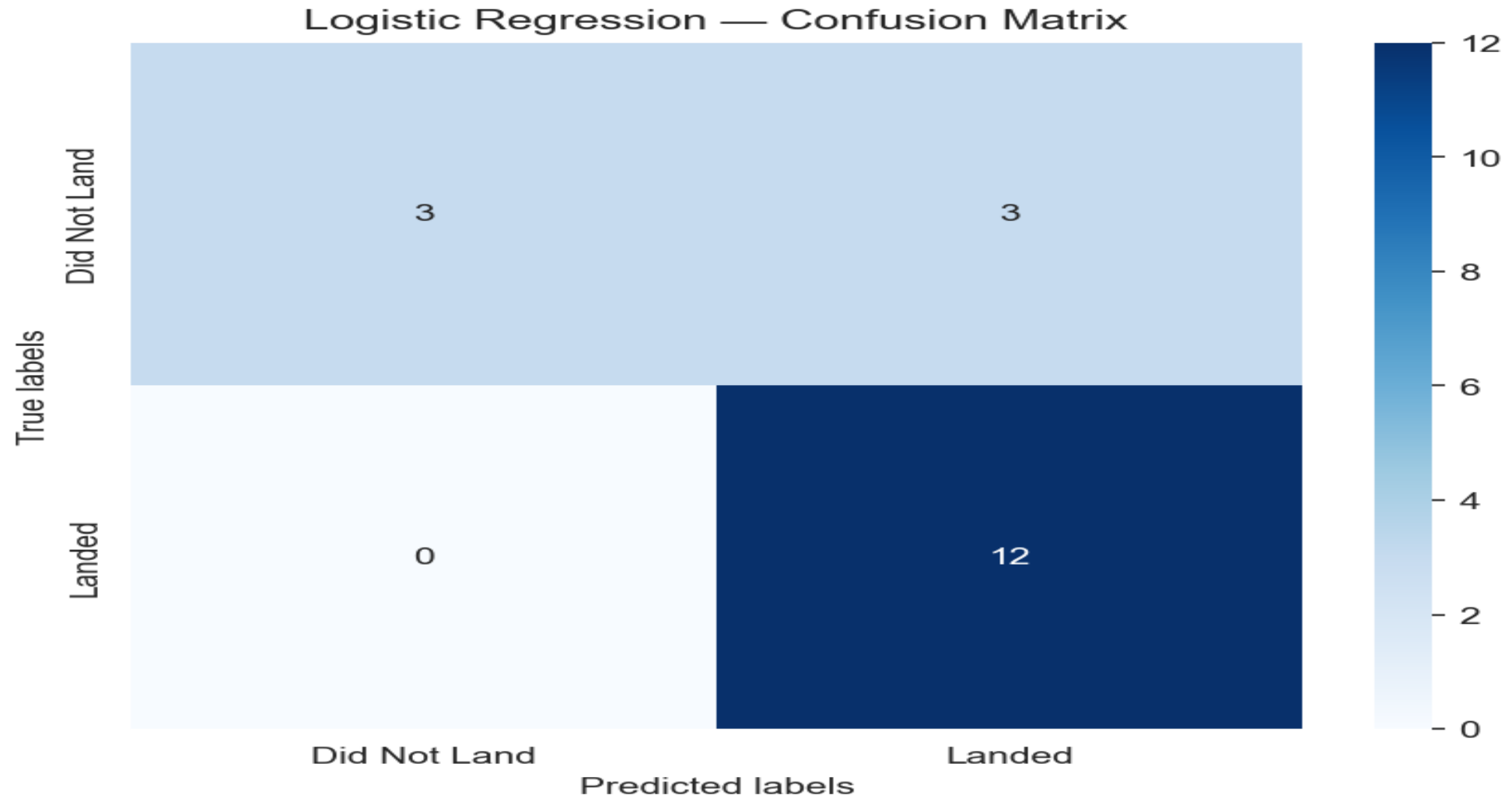
Scatter plot of Payload Mass vs Launch Outcome colored by Booster Version. FT and B5 boosters show higher success across payload ranges.
GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

Plotly Dash Results — Payload vs Outcome Scatter (KSC LC-39A)



KSC LC-39A: Successful launches across all payload ranges. B5 boosters dominate recent successful missions.
GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

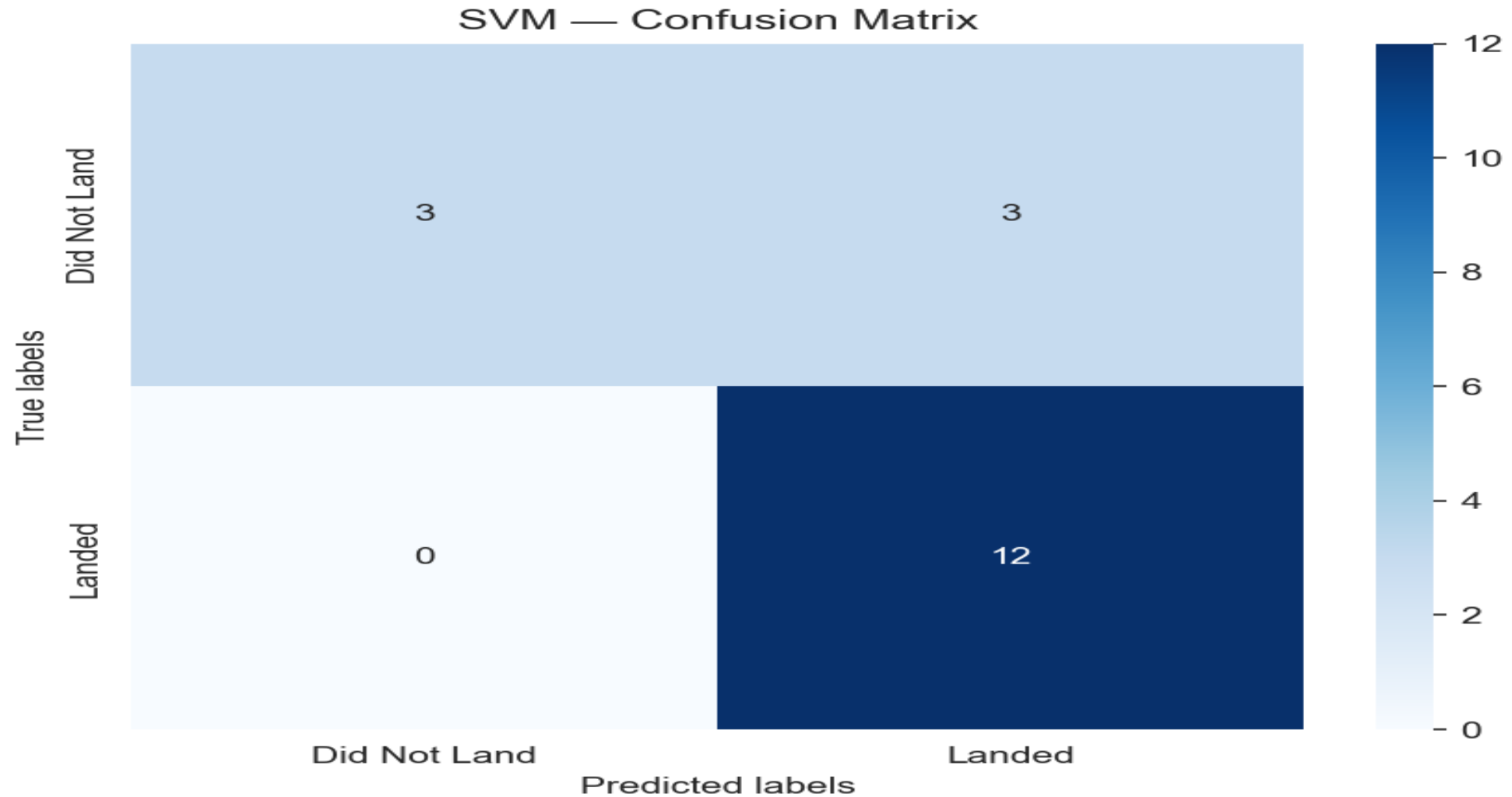
Predictive Analysis Results — Logistic Regression Confusion Matrix



Best params: C=0.01, penalty=l2, solver=lbfgs | CV Score: 84.64% | Test Accuracy: 83.33%

GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

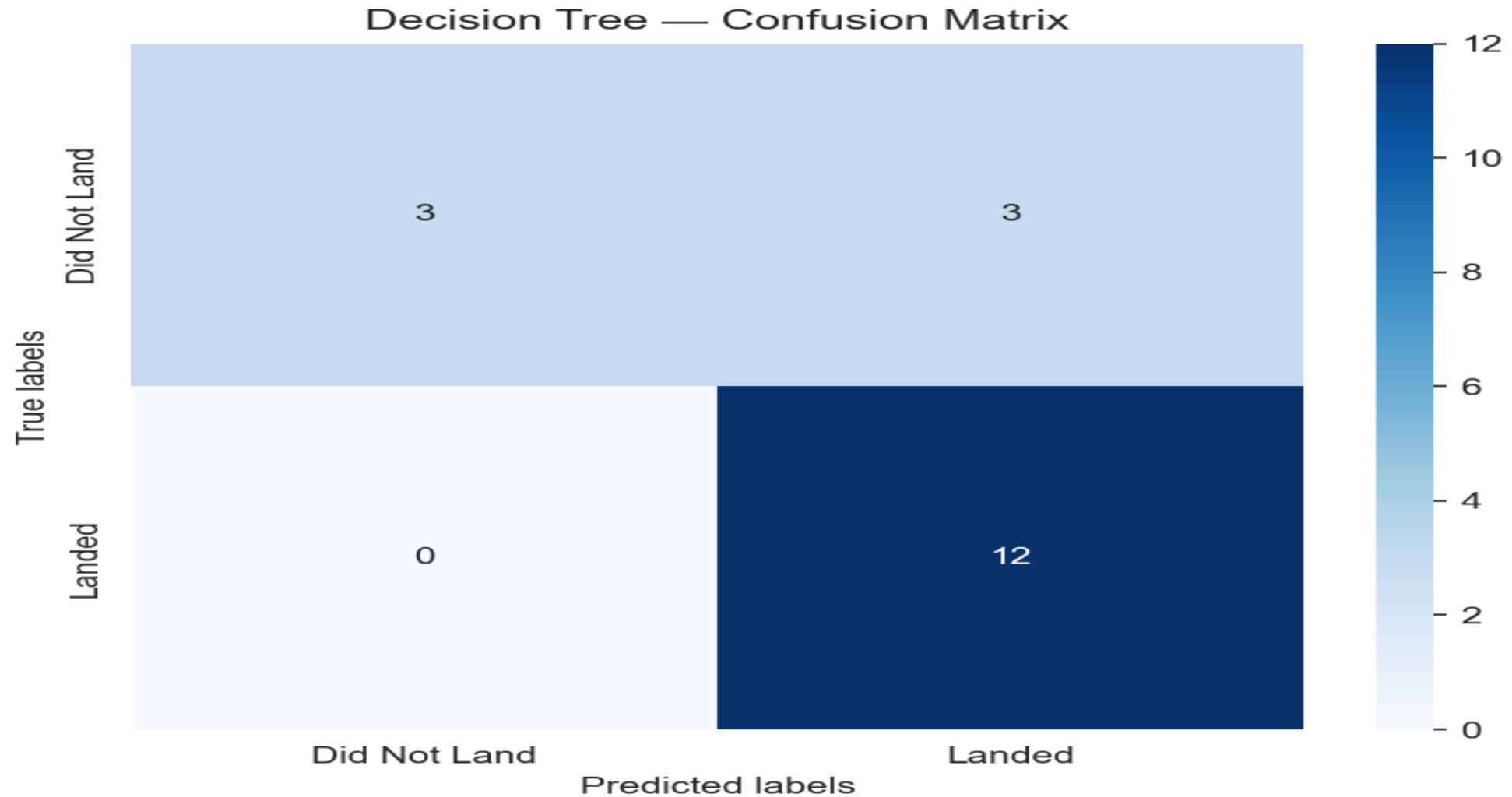
Predictive Analysis Results — SVM Confusion Matrix



Best params: kernel=sigmoid, C=1.0, gamma=0.032 | CV Score: 84.82% | Test Accuracy: 83.33%

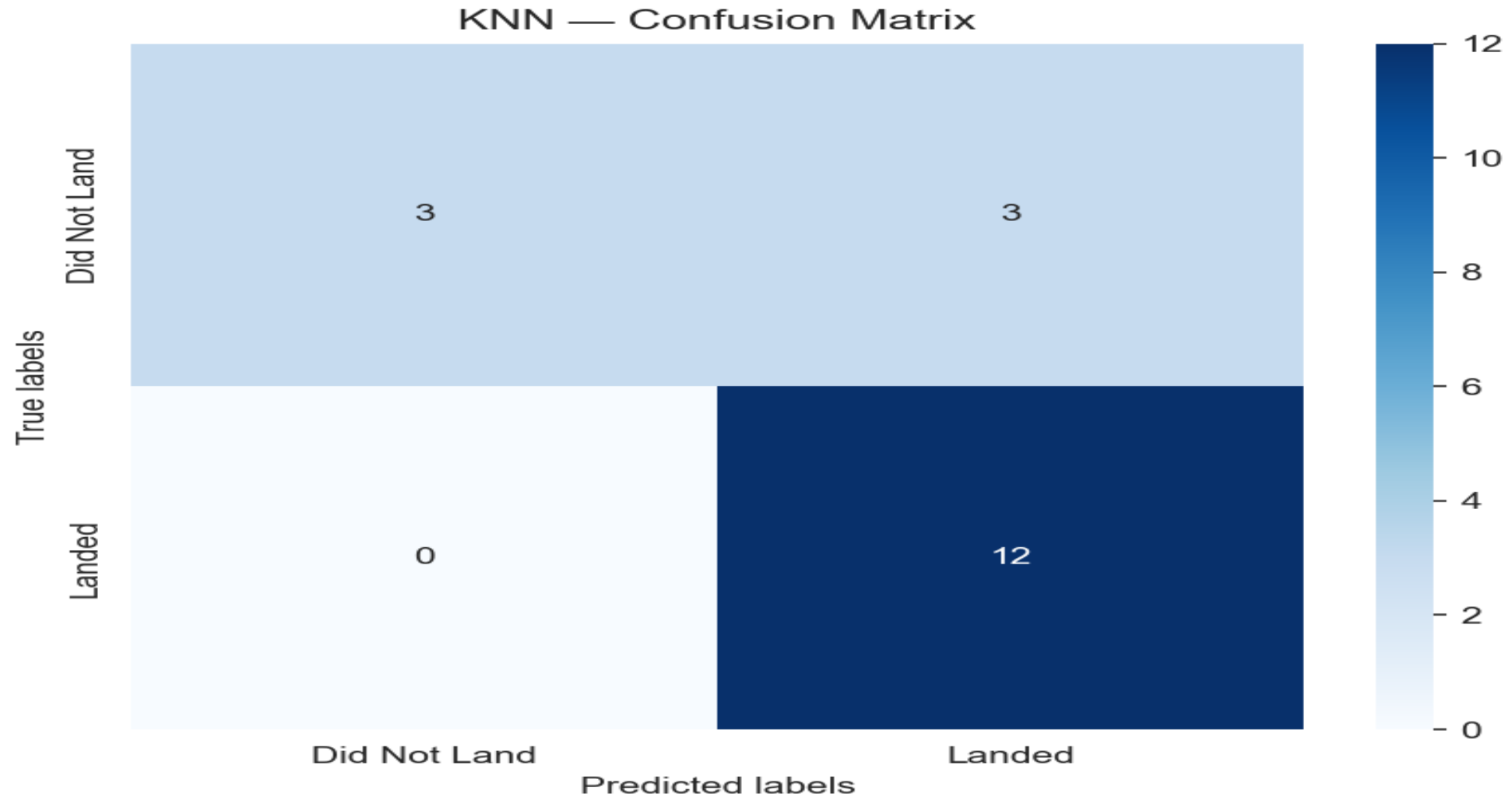
GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

Predictive Analysis Results — Decision Tree Confusion Matrix



Best params: criterion=entropy, max_depth=14, splitter=random | CV Score: 88.75% | Test Accuracy: 83.33%
GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

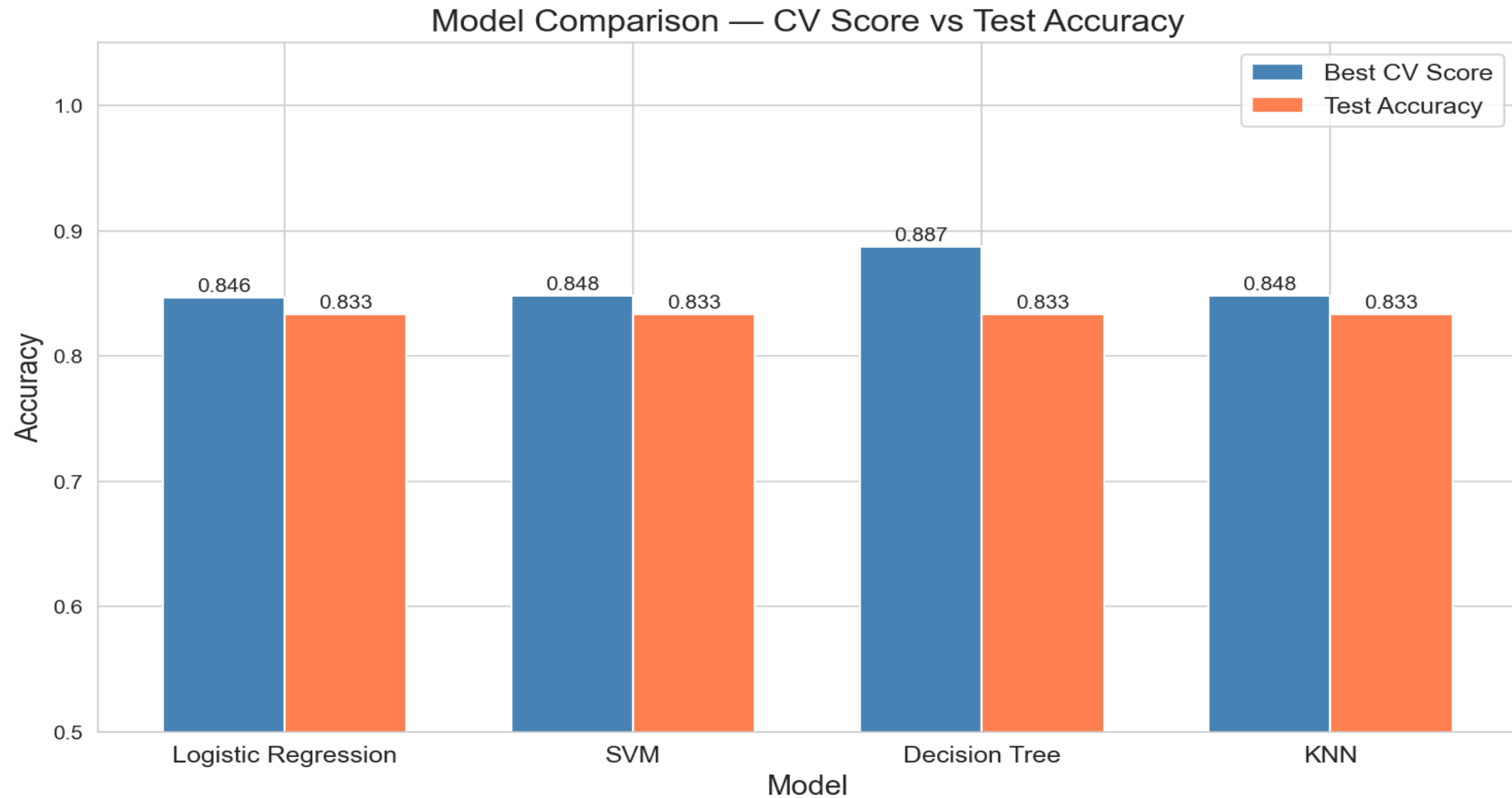
Predictive Analysis Results — KNN Confusion Matrix



Best params: n_neighbors=10, algorithm=auto, p=1 | CV Score: 84.82% | Test Accuracy: 83.33%

GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

Predictive Analysis Results — Model Comparison



All 4 models achieve 83.33% test accuracy. Decision Tree has the highest CV score (88.75%) and is recommended as the best model.

GitHub: <https://github.com/RyanOrdonez/spacey-capstone>

Predictive Analysis Results — Best Model & Evaluation

Model Comparison Summary:

Model	CV Score	Test Acc
Logistic Regression	84.64%	83.33%
SVM	84.82%	83.33%
Decision Tree	88.75%	83.33%
KNN	84.82%	83.33%

Best Model: Decision Tree

- Highest cross-validation score (88.75%)
- Best params: criterion=entropy, max_depth=14
- Easily interpretable (feature importance)
- All models tied on test accuracy (83.33%)
- Small test set (18 samples) limits differentiation

Key Evaluation Insights:

- Confusion matrices show most errors are false negatives (predicting failure when actual outcome was success)
- This is the safer error type for SpaceY: overestimating cost is better than underestimating it

Recommendations:

- Use Decision Tree as primary predictor
- Ensemble methods (Random Forest, XGBoost) could improve accuracy further
- Collect more data as SpaceX launches grow
- Feature importance analysis reveals Flight Number and GridFins as top predictors

Discussion

Findings Summary:

1. SpaceX landing success has improved dramatically from 0% (2010) to >80% (2020), driven by engineering iteration and Block 5 boosters.
2. Launch site matters: KSC LC-39A consistently outperforms other sites in success rate.
3. Orbit type is a strong predictor: LEO, ISS, SSO, ES-L1, GEO orbits have highest success.
4. Grid fins and landing legs are strong indicators of landing capability (enabled on later missions).
5. All ML models perform comparably (~83% accuracy) with Decision Tree showing highest CV performance.

Limitations:

- Small dataset (90 samples, 18 test) limits statistical power and model differentiation.
- Binary classification oversimplifies outcomes (controlled ocean landings classified as success).
- Data only covers 2010-2020; recent Starship developments may change competitive dynamics.

Future Work:

- Add weather data as features
- Use ensemble models (Random Forest, XGBoost)
- Time-series analysis of landing capability
- Cost-benefit analysis with prediction confidence

Conclusion & Innovative Insights

Conclusion:

We successfully built an end-to-end data science pipeline that predicts Falcon 9 first stage landing outcomes with 83.33% accuracy.

The Decision Tree classifier is recommended as the primary model (CV: 88.75%). This prediction enables SpaceY to estimate competitor launch costs and develop competitive pricing strategies.

Business Impact:

- **Predicted landing → bid ~\$62M (reusable)**
- **Predicted failure → bid ~\$165M (expendable)**
- **Savings per correct prediction: ~\$103M**

Innovative Insights:

1. Learning Curve Effect: SpaceX's landing

success follows a classic technology adoption S-curve, suggesting competitors can replicate this with sufficient investment and iteration.

2. Grid Fins as Innovation Marker: The

introduction of grid fins correlates strongly with landing success improvement, representing a key engineering breakthrough.

3. Geographic Strategy: All launch sites are

coastal and near the equator — a pattern SpaceY should replicate for its own launch facilities.

4. Payload-Orbit Optimization: Matching payload

mass to favorable orbits (LEO, SSO) significantly increases the probability of first stage recovery.

Appendix

GitHub Repository:

<https://github.com/RyanOrdonez/spacey-capstone>

Repository Structure:

```
spacey-capstone/  
├── data/      CSVs, SQLite database  
├── notebooks/ 7 Jupyter notebooks (01-07)  
├── scripts/   Helper scripts  
├── sql/       SQL query scripts  
├── images/    Charts & confusion matrices  
├── dash/      Plotly Dash application  
├── output/    Folium HTML maps  
├── slides/    PowerPoint & PDF  
├── requirements.txt  
└── README.md
```

Notebooks:

01_spacex_api_collection.ipynb
02_web_scraping.ipynb
03_data_wrangling.ipynb
04_eda.ipynb
05_sql_analysis.ipynb
06_interactive_visuals.ipynb
07_modeling.ipynb

Tech Stack:

Python, Pandas, NumPy, Matplotlib, Seaborn
Scikit-learn, BeautifulSoup, Requests
Folium, Plotly Dash, SQLite, Jupyter

Additional Charts:

- All 8 EDA charts saved in images/
- 4 confusion matrices in images/
- Model comparison bar chart in images/
- 4 Plotly Dash static charts in images/
- 3 Folium map visualizations in images/